SOLYTI

Jenkins - CI/CD

GitHub

Jenkins

SOMMAIRE

I. La philosophie Devops

II. Concept de livraison continue

III. Source control manager : GitHub

IV. L’orchestration CI/CD avec Jenkins

SOLYTI

I. La philosophie Devops

- Devops ?
- En quoi cela consiste ?
- Les avantages
- Les enjeux

La philosophie Devops

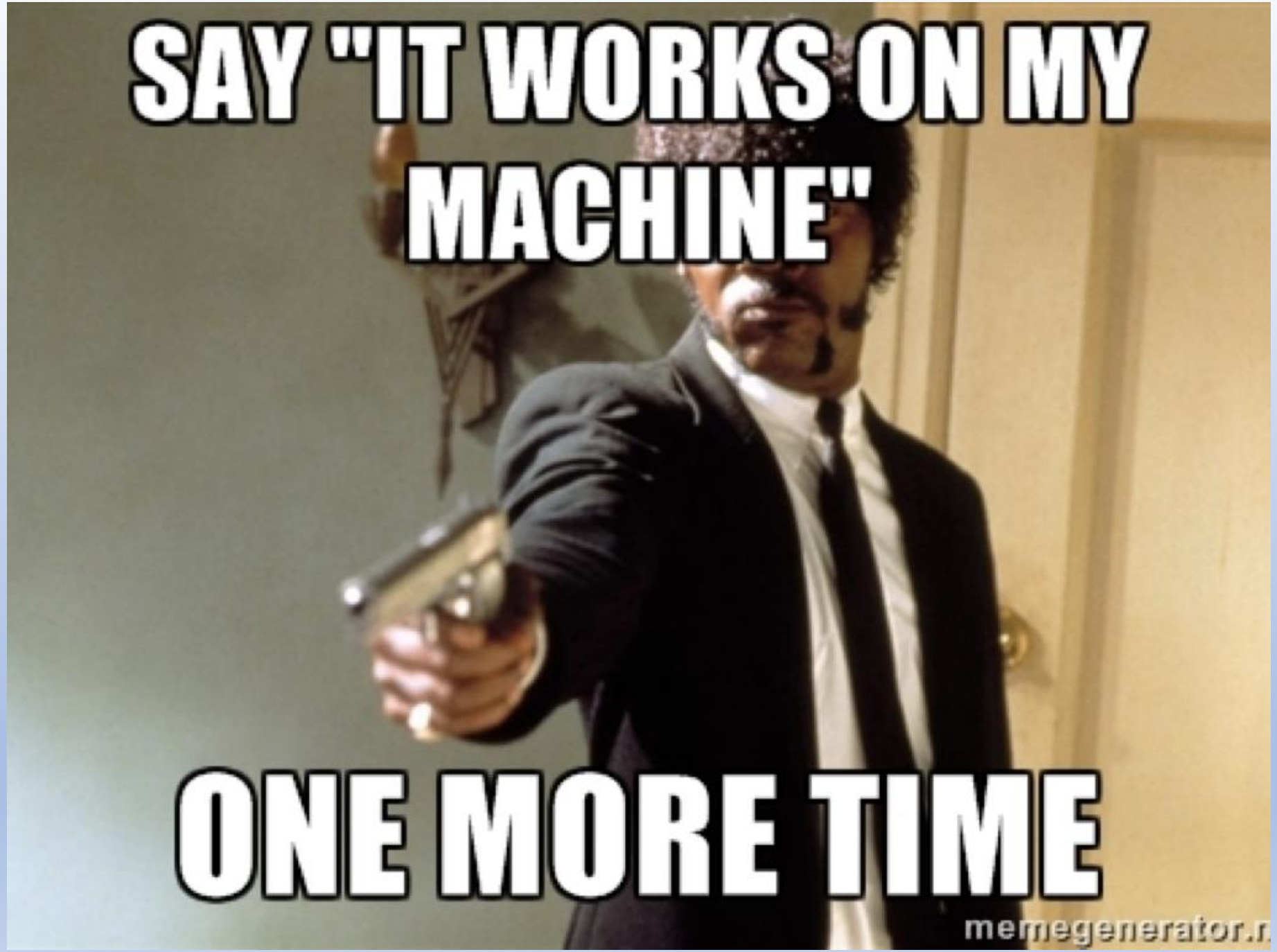
Devops ?

- Approche intégrée de développement de logiciels et d'exploitation informatique qui vise à améliorer la collaboration et la productivité entre les équipes de développement (Dev) et d'exploitation (Ops).

- Concept né de la nécessité de briser les silos traditionnels entre ces deux domaines, qui peuvent souvent entraîner des inefficacités, des erreurs et des retards dans la livraison des logiciels.

- On parle souvent de philosophie ou de culture devops.

WORKED FINE IN DEV
OPS PROBLEM NOW
memegen.com


SAY "IT WORKS ON MY MACHINE"
ONE MORE TIME
memegenerator.n

La philosophie Devops

En quoi cela consiste ?

De la collaboration et de la communication :

- Encourager la coopération étroite entre les développeurs, les testeurs, les administrateurs systèmes, et autres parties prenantes.
- Utiliser des outils de communication pour faciliter les échanges et la transparence.

De l'automatisation :

- Automatiser autant que possible les processus de développement, de test, de déploiement et de surveillance pour réduire les erreurs humaines et accélérer les cycles de livraison.

La philosophie Devops

En quoi cela consiste ?

De l’intégration Continue (CI) / déploiement Continu (CD) :

- Intégrer les modifications du code fréquemment.
- Automatiser les processus de test, de compilation, de stockage d’artefact et de déploiement pour que les applications puissent être mises en production rapidement et de manière fiable.

De la surveillance :

- Mettre en place une surveillance continue des applications et de l'infrastructure pour détecter rapidement les problèmes via des outils de journalisations et de monitoring.

La philosophie Devops

PAS DEVOPS

Compilation

- mvn install
- Tester localement

Release

- Changer les versions à la main
- mvn deploy
- Vérifier le bon déploiement dans le dépôt

Déploiements

- Récupération du paquet
- Transfert sur l'environnement de production
- Application de la configuration spécifique

Mise en place des surveillances

- Accès à l'outil de surveillance
- Déclaration de la nouvelle configuration

Suivi de projet

- Mise à jour des tickets
- Emails de confirmation

La philosophie Devops

DEVOPS

La philosophie Devops

Les avantages du devops

Livraison Rapide :

- Réduction du temps de cycle de développement, de test et de déploiement.
- Réduction du "time to market" pour les nouvelles fonctionnalités.

Amélioration de la Qualité :

- Détection précoce des erreurs grâce à des tests automatisés et une surveillance continue.
- Amélioration de la stabilité et de la fiabilité des applications.

La philosophie Devops

Les avantages du devops

Évolutivité / scalabilité:

•Capacité à gérer plus facilement les fluctuations de la demande grâce à des outils d'orchestration et de conteneurisation.

Innovation :

•Encouragement à l'expérimentation et à l'itération rapide.

•Création d'un environnement propice à l'innovation grâce à des cycles de feedback courts.

La philosophie Devops

Les enjeux et défis du devops

Culture et Changement Organisationnel :

•La mise en œuvre du DevOps nécessite souvent un changement de culture au sein de l'organisation -> Coûts / formations / adaptation

Complexité Technique :

•Gestion de la complexité accrue liée à l'intégration de nombreux outils et technologies -> compétences techniques avancées

Sécurité :

•Intégration de la sécurité dans le pipeline DevOps (DevSecOps) pour garantir des pratiques de sécurité cohérentes

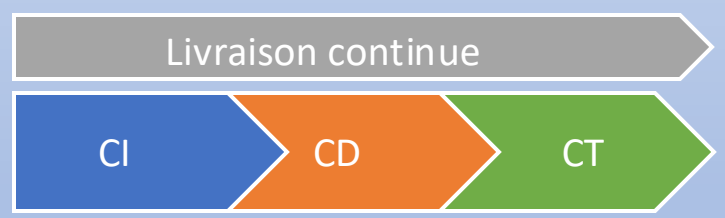
La philosophie Devops

En résumé

- Concaténation des 3 lettres « Dev » (développement) et « Ops » (exploitation).
- Culture / philosophie visant à améliorer la productivité des équipes et la fiabilité des infrastructures et applications.
- S’articule autour de technologies plus ou moins complexes mais toujours plus innovantes et nombreuses.

II. Concept de livraison continue

- Livraison continue ?
- Intégration continue
- Déploiement continu
- Tests continus



Concept de livraison continue

Livraison continue ?

Approche qui permet de déployer des applications de manière fiable et rapide, en assurant que chaque modification de code est toujours prête pour la production.

Elle intègre 3 concepts :

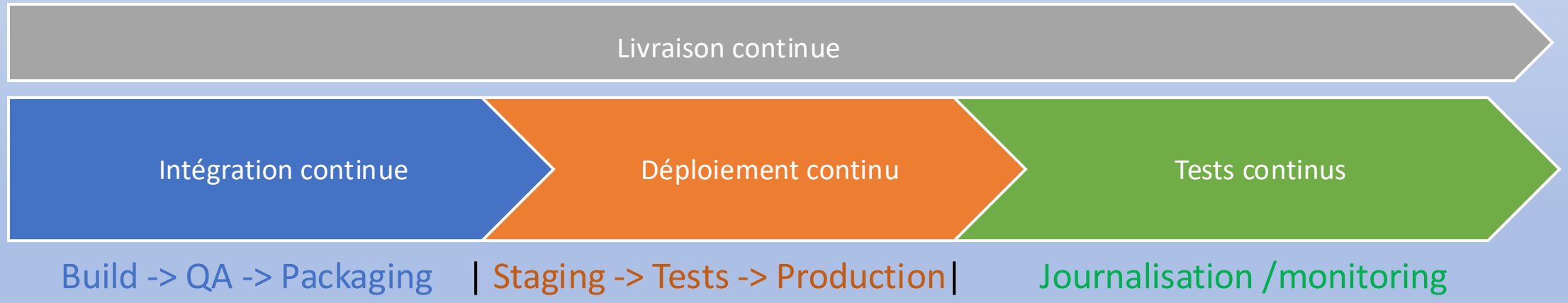
- Intégration continue
- Déploiement continu
- Tests continus

Concept de livraison continue

Livraison continue ?

La livraison continue repose sur l'offre pipeline CI/CD.

C'est une déclaration de processus complexes à effectuer pour livrer une application en production rapidement et avec la meilleur fiabilité



Concept de livraison continue

Intégration continue - CI

Ensemble de bonnes pratiques et d'outils (briques) qui permettent de s'assurer de la qualité / évolutivité d'une application.

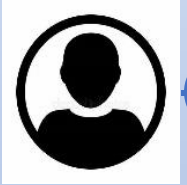
Permet une accélération des releases tout en réduisant le nombre d'erreurs.

Principales composantes :

- Centralisation et partage du code source (SCM)
- Persistance régulière du code applicatif (Workflows et versionning)
- Automatisation des builds et des tests (QA & security)

Concept de livraison continue

Intégration continue - CI



1

Le code source

Hébergé sur un dépôt centralisé dans un gestionnaire, permet de versionner son code et facilite le travail collaboratif

2

Le Job

Permet d'assurer la réalisation des actions de build de façon automatisée à l'aide de scripts d'exécution

3

Le build

Permet la construction du binaire

4

Les dépendances

Permet de récupérer les dépendances technologiques nécessaires au bon fonctionnement de l'application

5

Les binaires

Permet le stockage du binaire après sa construction

Concept de livraison continue

Les briques de la CI

La gestion des codes sources

- Développement collaboratif
- Gestion des versions
- Synchro locale et serveur

L'orchestration de l'intégration

- Automatisation des tâches
- Intégration des outils

L'outillage de build

- Automatisation du build
- Génération des packages

La gestion des binaires

- Stockage des binaires
- Stockage des images
- Gestion des dépendances

La vérification qualité / sécurité

- Tests de qualité
- Tests de sécurité

Concept de livraison continue

Déploiement continu - CD

Pratique qui permet de déployer des applications en production de manière fiable et rapide, en s'assurant que chaque modification de code peut être mise en production à tout moment.

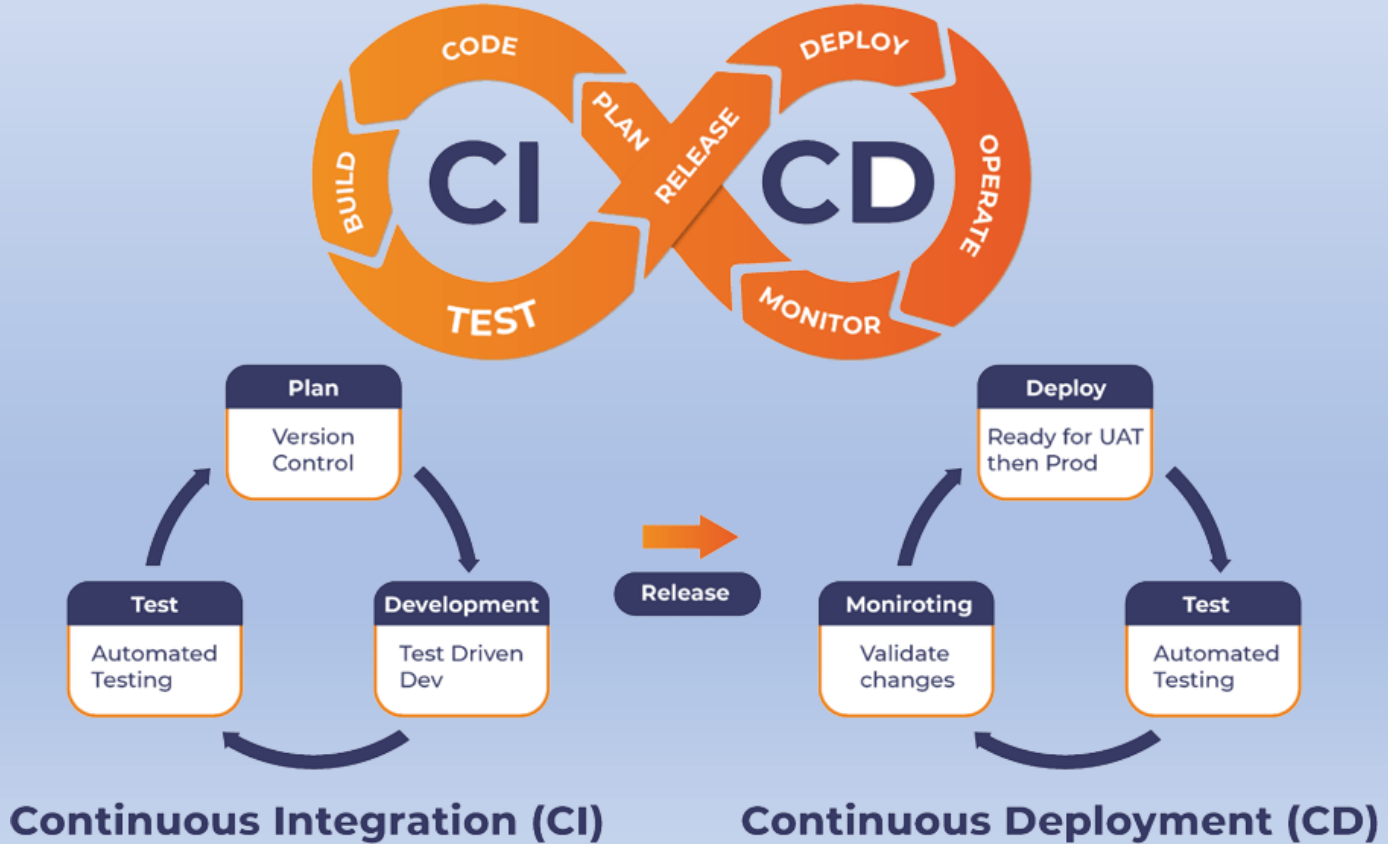
Principales composantes :

- Environnement cloisonnés mais similaires / identiques
- Tests (intégration, end to end…)
- Automatisation des déploiements

Concept de livraison continue

Déploiement continu - CD

Une fois que la construction et les tests qualités d'une application sont validés par les briques d'intégration continue, la CD prends la suite.

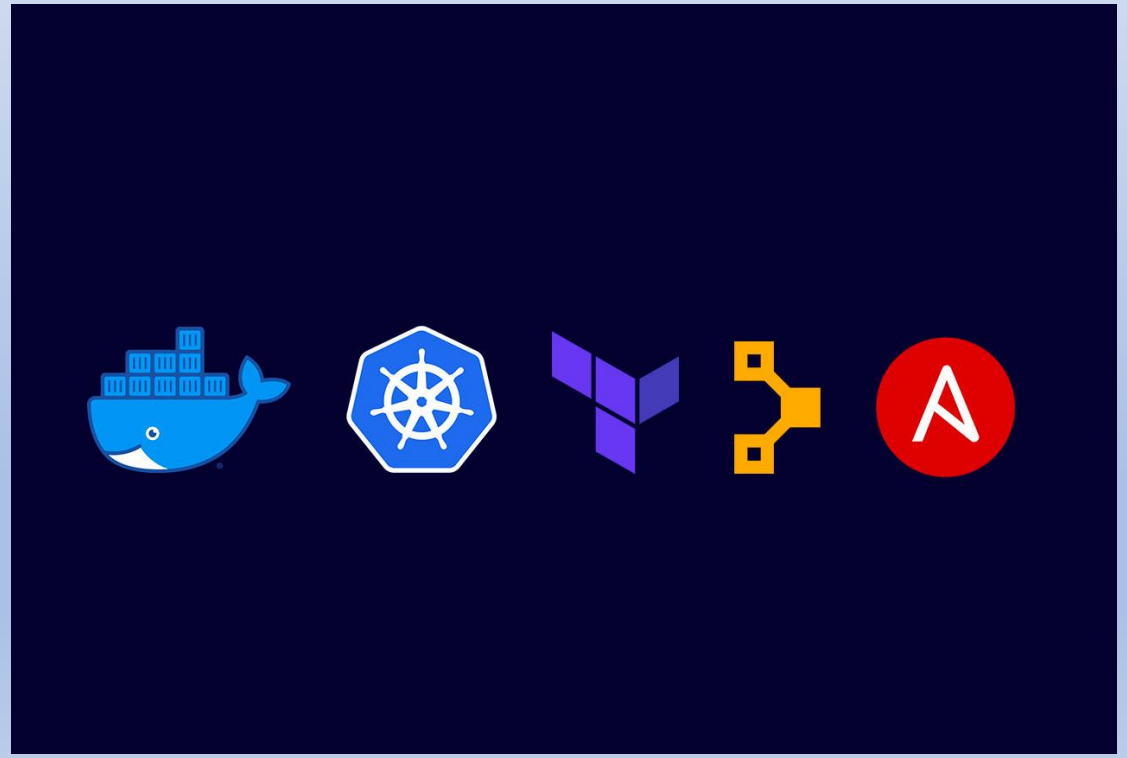


Concept de livraison continue

Les briques de la CD

On retrouve différents outils pour gérer la partie déploiement :

- Instance Cloud /Docker / Kubernetes,
- Terraform / Opentofu,
- Ansible / Puppet / Chef.



Concept de livraison continue

Tests continus - CT

Ils consistent en l'automatisation et l'exécution fréquente de tests spécifiques tout au long du cycle de développement.

Ils sont présents dans différentes phases d'un pipeline CI/CD.

Principales composantes :

- Automatisation des tests (unitaires, intégrations, UAT, securité,…),
- Surveillance, journalisation et reporting.

Concept de livraison continue

Tests continus - CI

Tests pendant la phase d'intégration continue (CI) :

- Tests unitaires : Vérifient que chaque composant individuel du code fonctionne correctement.
- Analyse statique du code : Utilisation d'outils pour vérifier la qualité du code et détecter les erreurs potentielles.
- Tests de sécurité : Identifient les vulnérabilités potentielles et s'assurent que l'application est sécurisée.

Concept de livraison continue

Tests continus - CD

Tests pendant la phase de livraison continue (CD) :

- Tests d'intégration : Vérifient que les différents modules ou composants interagissent correctement entre eux.
- Tests de bout en bout (E2E) : Simulent des scénarios réels pour s'assurer que l'application fonctionne du point de vue utilisateur.
- Tests de performance : Évaluent la performance de l'application sous différentes charges de travail.

Concept de livraison continue

Tests continus - Les outils de test

CI	Tests unitaires	JUnit, NUnit, pytest, Jest
	Analyse statique du code	SonarQube, ESLint, Pylint, Checkstyle
CD	Tests d'intégration	Testcontainers, Postman, Spring Test
	Tests de régression	Frameworks unitaires et d'intégration avec suites spécifiques
	Tests de bout en bout (E2E)	Selenium, Cypress, Robot Framework, Playwright
	Tests de performance	JMeter, Gatling
	Tests de sécurité	OWASP ZAP, Burp Suite
POST	Monitoring des performances	Prometheus, Grafana, ELK Stack, New Relic, Datadog
	Monitoring de la sécurité	Prometheus, Grafana, ELK Stack, New Relic, Datadog

III. Source control manager

- SCM ?
- En quoi cela consiste ?
- Introduction a Github
- Workflows

GitHub

Source control manager

SCM ?

Un SCM pour source control manager est un système qui permet de gérer les modifications apportées au code source d'un projet logiciel

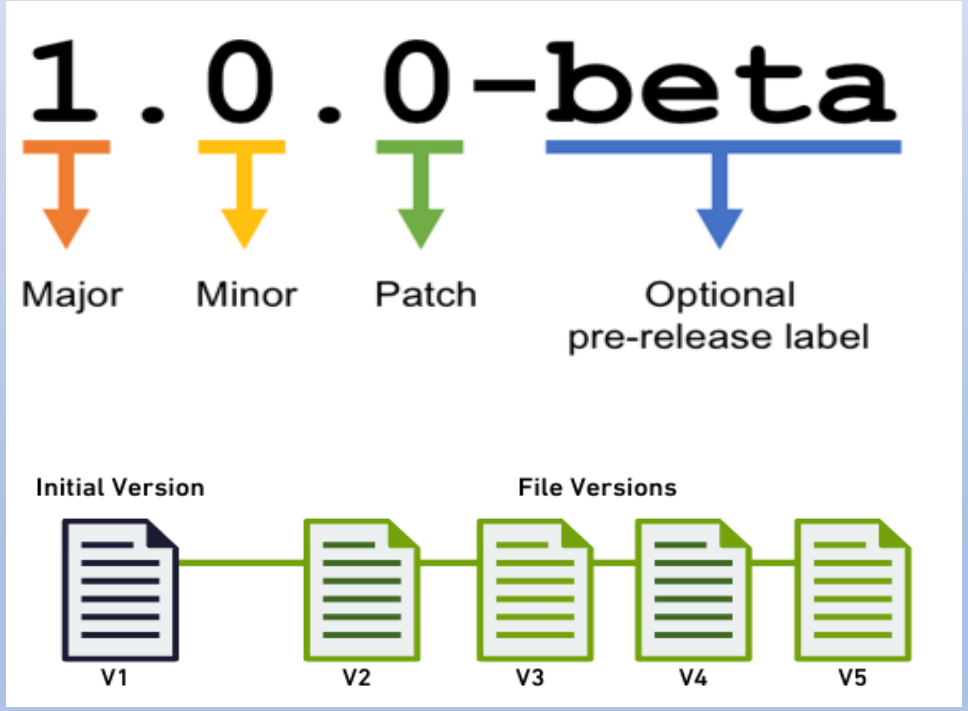
- Point central du code source des applications,
- Permet le suivi des modifications,
- Organisation du process de développement (workflows),
- Outils collaboratif.

Source control manager

En quoi cela consiste

Suivi des modifications :

- Un SCM enregistre chaque modification apportée au code source, permettant de suivre qui a fait quoi et quand.
- Il maintient un historique complet des modifications, facilitant ainsi le retour à des versions antérieures du code si nécessaire.

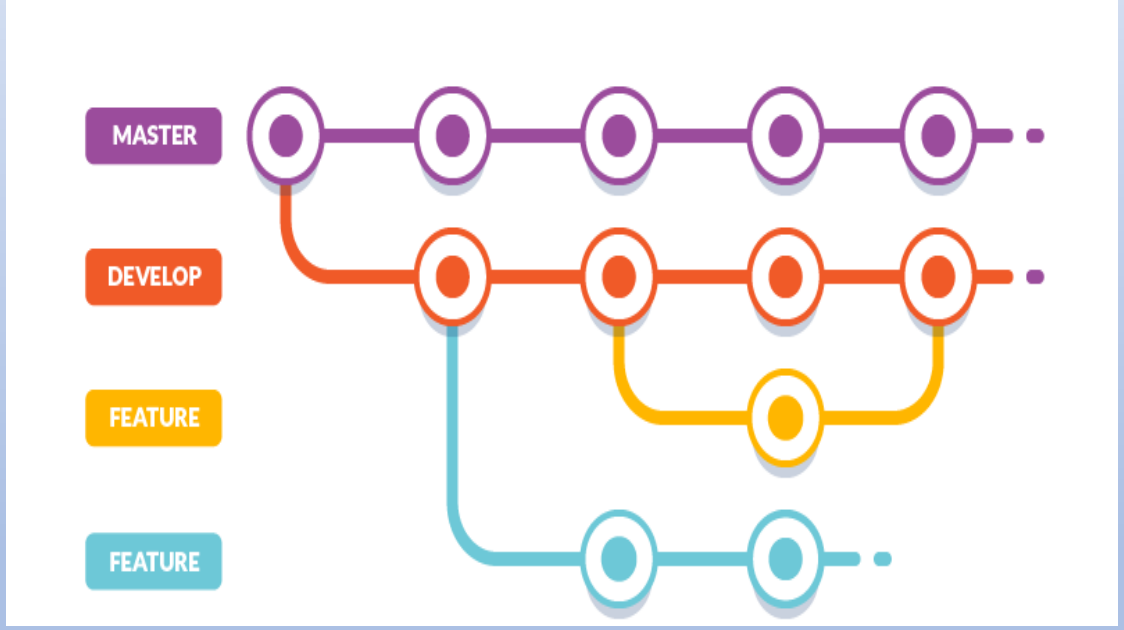


Source control manager

En quoi cela consiste

Gestion des versions :

- Il permet de créer différentes versions (ou branches) du code source, facilitant ainsi le développement parallèle de nouvelles fonctionnalités, la correction de bugs et la gestion des versions de production.



Source control manager

En quoi cela consiste

Collaboration :

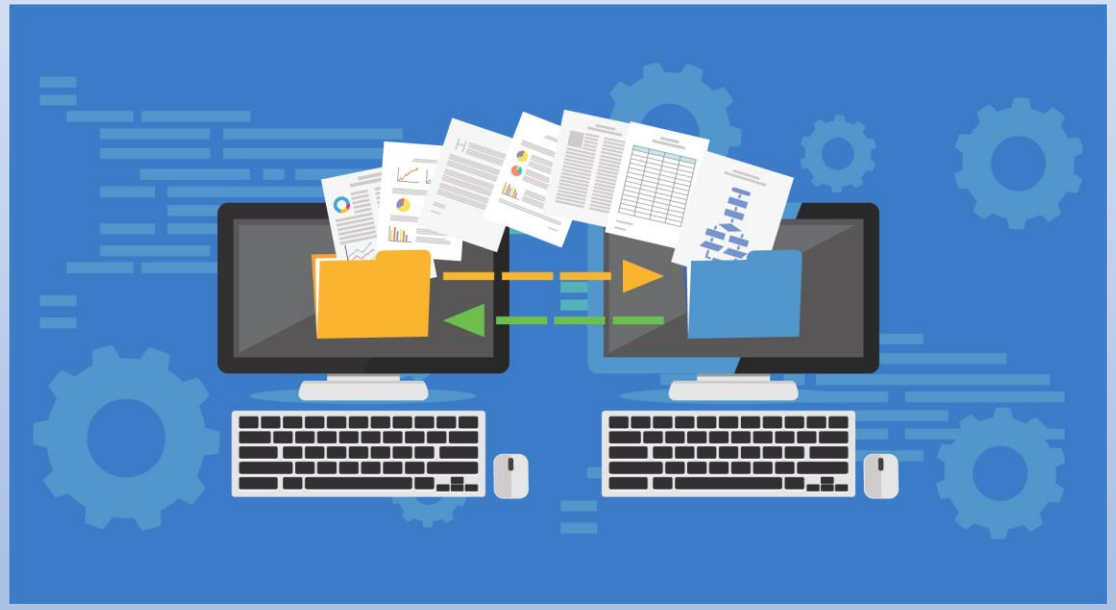
- Les SCM permettent à plusieurs développeurs de travailler simultanément sur le même projet sans interférer les uns avec les autres.
- Ils gèrent les conflits de fusion lorsque plusieurs développeurs modifient les mêmes parties du code.

Source control manager

En quoi cela consiste

Sauvegarde et récupération :

- En conservant une copie historique de tout le code source, un SCM sert de sauvegarde en cas de perte de données.
- Il permet de récupérer facilement des versions précédentes du code si nécessaire.



Source control manager

Introduction à GitHub

GitHub est une plateforme en ligne de développement collaboratif basée sur le système de version git.

Git : Système de contrôle de version créer par Linus Torvald.

GitHub offre des outils de collaboration, de revue de code, et d'intégration continue.

Source control manager

Introduction à GitHub – Système Git

Exemple de commandes git :

- *clone : Fait une copie locale d'un dépôt*
- *fetch :* Récupère les informations distantes
- commit : Enregistre les modifications sur la copie locale
- *push : Transmet les modifications apportées à un dépôt*
- *pull : Récupère les modifications apportées à un dépôt sur la copie locale*
- *tag : Ajoute un tag au dépôt local*

Source control manager

Introduction à GitHub - Objectif

Objectif de GitHub : fournir un environnement de développement cohérent implémentant des concepts qui permettent de sécuriser un projet et de collaborer.

Ses fonctionnalités principales :

- Repositories
- Branches
- Commits
- Pull requests
- Issues

Source control manager

Introduction à GitHub – Notions et fonctionnalités

Repositories: Stockage centralisé du code source d'un projet avec historique des modifications.

Branches: Permet de travailler sur différentes versions du code en parallèle (cf workflows).

Commits: Enregistre les changements effectués dans le code avec un message descriptif.

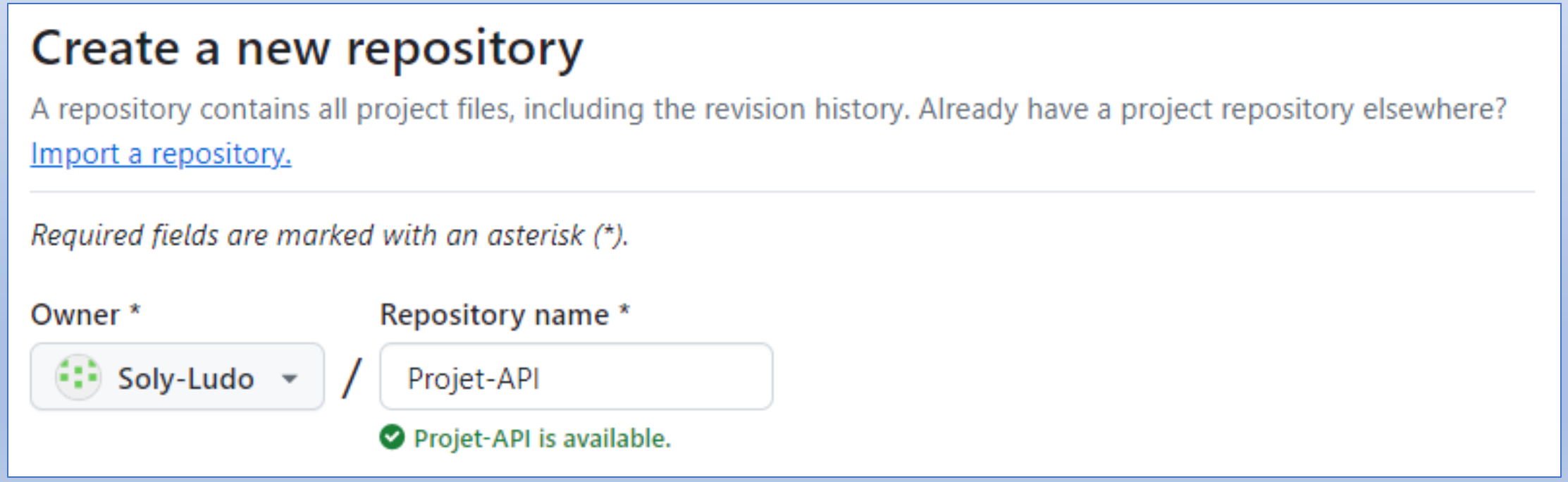
Pull Requests: Propose des modifications dans une branche et demande une revue avant fusion.

Issues: Suivi des bugs, tâches, et nouvelles fonctionnalités.

Source control manager

Introduction à GitHub – Création d'un repository

Un repository appartient à son créateur et inclus un nom qui doit être unique au sein du repository GitHub.

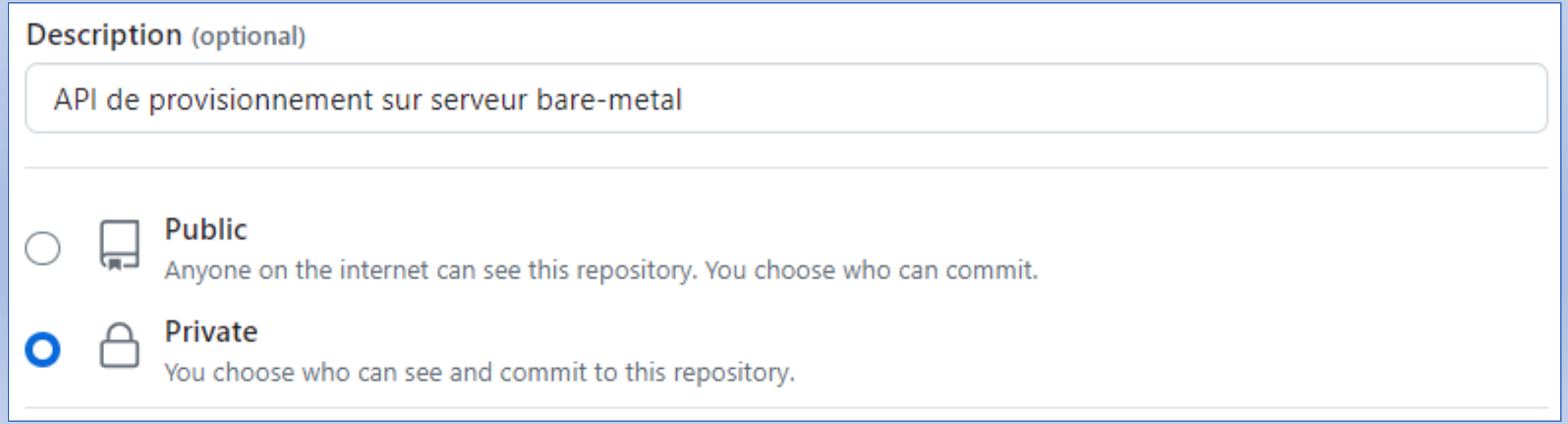


Source control manager

Introduction à GitHub – Visibilité d'un projet

Public : Tous le monde peut voir, cloné ou télécharger le code source sans avoir de compte sur GitHub. (Portfolios, projet open source…).

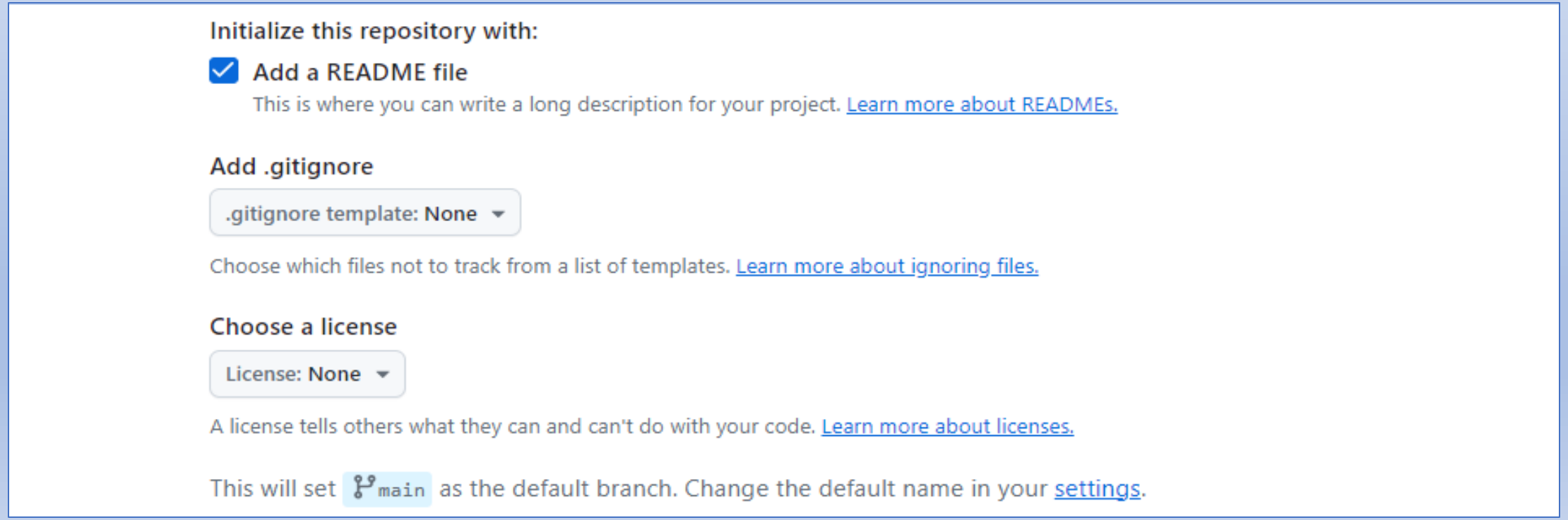
Private : visible que par le propriétaire du dépôt et les collaborateurs explicitement ajoutés. Les utilisateurs doivent être invités pour accéder au dépôt.



Source control manager

Introduction à GitHub – Création d'un projet GitHub

On peut inclure un fichier README.md pour initialiser le repository et choisir un template de .gitignore en fonction des technologies ainsi qu'une license.



Exercice 1

Accéder à GitHub et créer un repository

Créer et alimenter son projet GitHub

- Création et/ou access au compte GitHub
- Création d’un repository (projet)
- Installer un IDE
- Cloner le projet en local
- Alimenter son projet

Source control manager

Introduction à GitHub – Workflows

Dans un contexte général des SCM, le terme workflow fait référence à une organisation de travail.

C’est un « modèle d’organisation » qui se met en place sur un projet pour définir les stratégies et méthodologies pour organiser et gérer le développement du code source.

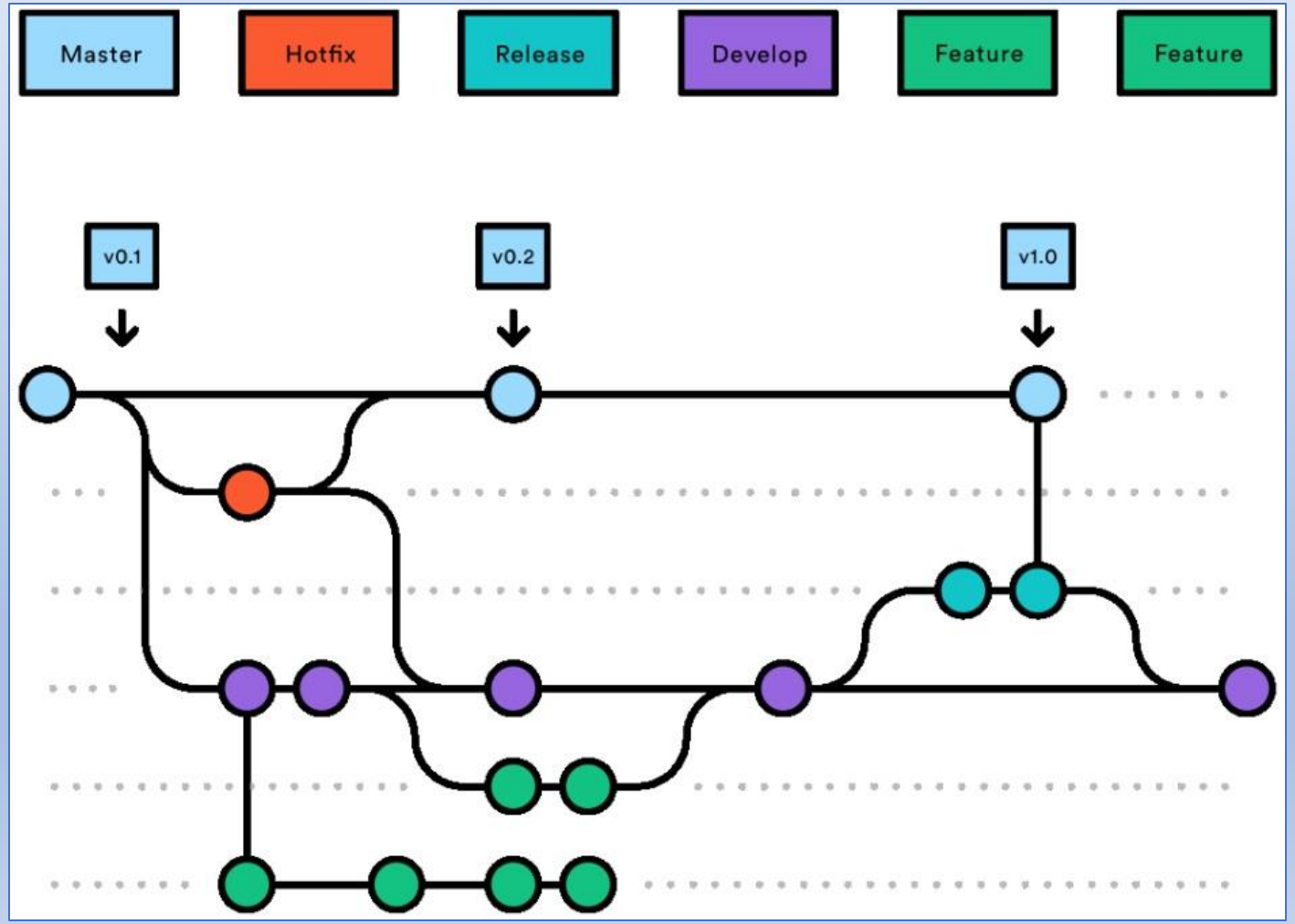
Il existe différents workflows quand on parle d’organisation de développement de code sur un SCM.

Source control manager

Introduction à GitHub – Gitflow

Gitflow est un workflow éprouvé et adopté dans de nombreux projet de développement pour sa rigueur dans l'organisation des branches.

Il comporte 5 « types de branches » avec leur caractéristique et spécificité



Source control manager

Introduction à GitHub – Workflows

Attention :

Dans le contexte de GitHub action, un workflow est aussi une configuration automatisée pour définir des pipelines de CI/CD en utilisant des fichiers de configuration basés sur du code.

- Ces fichiers sont écrit en yaml
- Ils contiennent des « jobs » qui peuvent être exécutés en parallèle ou séquentiellement.
- Chaque job peux contenir une ou plusieurs étapes (steps) qui sont des tâches réelles à exécuter.

Source control manager

Introduction à GitHub – Pull requests

Les pullrequests (PR) sont un élément central de la collaboration sur GitHub.

Elles permettent aux développeurs de proposer des modifications, de demander des retours et d'implémenter des révisions avant d'intégrer les changements dans la branche principale d'un projet.

Source control manager

Introduction à GitHub – Pull requests

Principales utilités :

- **Proposition** :

Une pull request propose des modifications apportées à une branche spécifique (souvent une branche de fonctionnalité ou de correction de bug) pour qu'elles soient intégrées dans une autre branche (généralement develop ou main)

- **Revue de Code** :

Les pull requests sont un moyen de demander des retours sur le code, permettant à d'autres contributeurs de revoir, commenter et suggérer des améliorations.

Source control manager

Introduction à GitHub – Pull requests

Principales utilités :

- **Echanges dans l'équipe** :

Les pull requests facilitent les discussions autour des changements proposés. Les contributeurs peuvent poser des questions, signaler des problèmes et fournir des conseils.

- **Discussions intégrées** :

Les conversations dans les pull requests permettent de résoudre les problèmes de manière collaborative avant que les changements ne soient fusionnés.

Source control manager

Introduction à GitHub – Pull requests

Principales utilités :

- **Tests Automatisés** :

Elle peuvent déclencher des workflows CI/CD pour exécuter des tests automatisés, vérifier la qualité du code et s'assurer que les modifications n'introduisent pas de bugs.

- **Fusion** :

Une fois les révisions terminées et les tests passés, les modifications peuvent être fusionnées dans la branche cible.

Source control manager

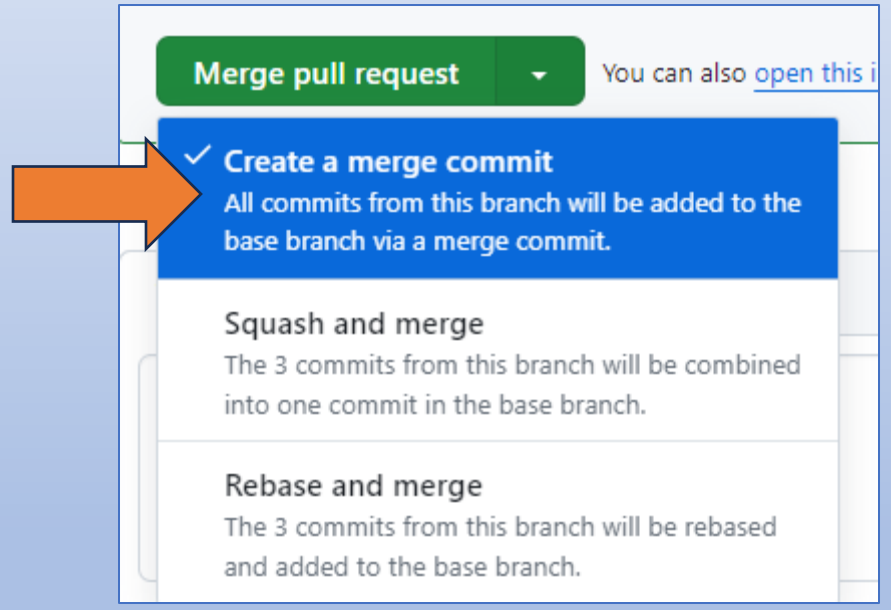
Introduction à GitHub – Pull requests

Les types de merges : Merge commit

- Cette méthode crée un commit de fusion lorsque deux branches sont fusionnées.

- Elle combine les modifications de la branche source dans la branche cible tout en conservant l'historique de chaque commit.

- Il montre explicitement qu'il y a eu une fusion, ce qui est utile pour suivre les changements dans des projets complexes ou collaboratifs.



Source control manager

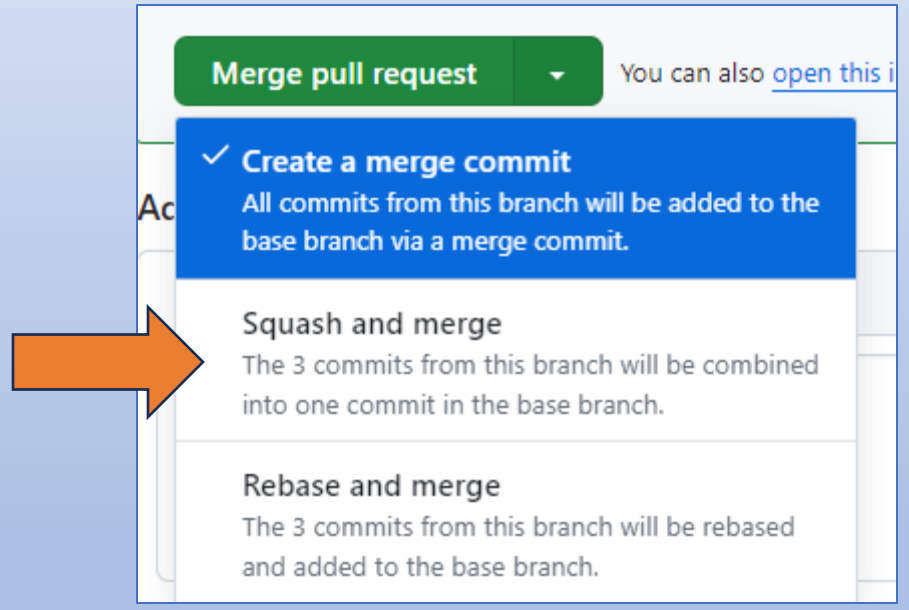
Introduction à GitHub – Pull requests

Les types de merges : Squash and merge

- Cette méthode combine tous les commits de la branche source en un seul commit avant de fusionner.

- Elle squash tous les commits de la branche source en un seul commit, puis applique ce commit unique sur la branche cible.

- Utile pour éviter d'encombrer l'historique avec de nombreux petits commits, surtout lorsque les commits intermédiaires ne sont pas nécessaires individuellement.



Source control manager

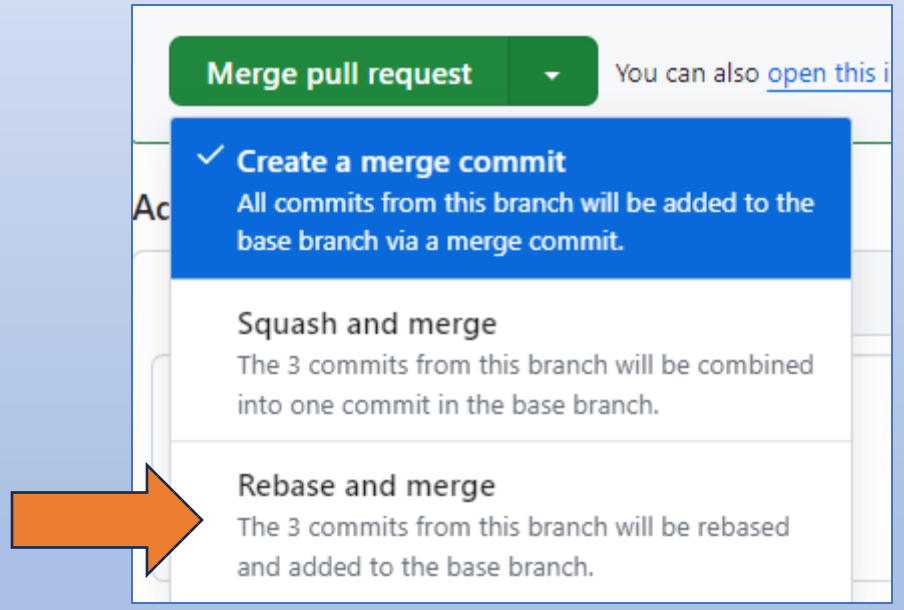
Introduction à GitHub – Pull requests

Les types de merges : Rebase and merge

- Cette méthode réapplique les commits de la branche source sur la branche cible comme s'ils étaient créés à partir de cette dernière.

- Au lieu de créer un commit de fusion, Git réécrit l'historique des commits de la branche source pour les placer après le dernier commit de la branche cible.

- Utile pour garder un historique linéaire et lisible, idéal pour les projets où la clarté de l'historique est importante.



Exercice 2

Création de branches sur un modèle Gitflow et pull requests

Gitflow et pull requests

- Creation de branche sur le modèle gitflow
- Alimentation des Nouvelles branches
- Creation de pull request

III. Orchestration CI/CD avec Jenkins

- Jenkins ?
- En quoi cela consiste ?
- Introduction a Jenkins

Jenkins

Jenkins

Jenkins ?

Outil open-source d'intégration continue et de livraison continue (CI/CD).

Son but :

Automatiser les tâches répétitives liées au développement et à la livraison de logiciels.

Il est principalement utilisé pour construire, tester et déployer des applications de manière continue, facilitant ainsi un cycle de développement plus rapide et fiable.

Jenkins – bref historique

Fondé par Kohsuke Kawaguchi, Jenkins était à l'origine nommé Hudson et fut renommé en 2011 après des différents entre son auteur et Oracle qui avait fait un fork du projet et revendiquait les droits sur le nom du projet. Hudson n'est plus maintenu à jour et a été annoncé obsolète en février 2017

Multiplement récompensé :

- Meilleur projet open source DevOps 2017
- Meilleur projet open source DevOps 2016
- Bossie Award 2014
- Geek choice Award 2014
- Duke's choice Award 2008 (catégorie 'Developer Solution')

Jenkins – En quoi cela consiste

Jenkins permet d'orchestrer toutes les actions de CI/CD et a l'avantage d'être :

- Open Source et gratuit
- Paramétrable via l'utilisation de plus de 1500 plugins
- Fédérateur d'une vaste communauté de développeurs
- Portable sur toutes les principales plateformes logicielles
- Interconnectable avec la plupart des gestionnaires de sources, des outils de build, de déploiement et tests

Jenkins – En quoi cela consiste

Automatisation des Tâches :

- Jenkins permet d'automatiser les tâches, réduisant ainsi les erreurs humaines et accélérant le processus de livraison.
- Celles-ci peuvent inclure la compilation du code, l'exécution de tests, le packaging d'applications, le déploiement sur des serveurs, etc.

Plugins :

- Jenkins dispose d'une architecture extensible avec de nombreux plugins (plus de 1 500 plugins) qui permettent de l'intégrer à d'autres outils tels que Git, Docker, Maven, Gradle, et bien d'autres.

Jenkins – En quoi cela consiste

Pipeline as Code :

- Jenkins permet de définir des pipelines en tant que code en utilisant des fichiers Jenkinsfile, écrits en syntaxe Groovy.
- Les pipelines peuvent être simples ou complexes, incluant des étapes parallèles, des conditions, des boucles, etc.

Interface Web :

- Jenkins fournit une interface web intuitive pour configurer les tâches, gérer les builds, surveiller les pipelines et visualiser les résultats des tests.
- Les utilisateurs peuvent créer, configurer et surveiller leurs jobs via cette interface.

Jenkins – En quoi cela consiste

Intégration Continue (CI) :

- Jenkins interagit avec les SCM et déclenche des builds automatiques lorsque de nouvelles modifications sont détectées.
- Il exécute des tests unitaires et d'intégration pour s'assurer que les modifications n'introduisent pas de bugs.

Déploiement Continue (CD) :

- Jenkins peut déployer automatiquement les applications sur des environnements de test, de staging ou de production après un build réussi.
- Il supporte diverses stratégies de déploiement, telles que le déploiement bleu/vert, les déploiements canaris, etc.

Jenkins

Jenkins – Introduction à Jenkins

Le fonctionnement de Jenkins s'articule autour de l'utilisation de plugins et de jobs qui permettent d'orchestrer les différents outils associés à la livraison applicative.

Concept et vocabulaire associé :

- Job (item) : un processus de « build »
- Build : une exécution d'un job
- Workspace : espace dédié à une exécution d'un job
- Artefact : Application compilées ou image docker de l'application
- Webhook : Requête HTTP qui permet de déclencher les jobs

Jenkins – Introduction à Jenkins

Concept et vocabulaire associé :

- Pipeline : Un type de job permettant de décrire un processus complexe
- Pipeline multi-branche : Une organisation pipeline différenciant le process de build par branche du dépôt de source.
- Credentials : Secrets utilisables par Jenkins lors des builds (identifiants de connexion à github, token d'authentification API….)
- Master/Main : Instance principal qui ordonnance les traitements
- Slaves/Agent : Environnement d'exécution des traitements
- Polling(SCM) : Jenkins envoi des requêtes au SCM pour connaître les changements

Exercice 3

Installation et découverte de Jenkins

TP3 - Installation et découverte de Jenkins

- Déployer un serveur Jenkins
- Découvrir l’interface et les différents onglets

Jenkins – Introduction à Jenkins

Exemple d'étapes réalisées par un pipeline Jenkins « from SCM » dans un processus CI :

- Un push ou une merge request sont réalisés sur le **SCM.**
- Jenkins va créer un répertoire dédié (workspace) à l'exécution du pipeline et cloner le projet git entièrement
- Il lance ensuite l'environnement nécessaire à la compilation du binaire (ex un conteneur maven + dependances pour compiler du java) et compile le code

Jenkins – Introduction à Jenkins

Suite des étapes réalisées par un pipeline Jenkins « from SCM » dans un processus CI

- Jenkins effectue ensuite des tests unitaires / de sécurité via des frameworks et outils de test
- L'artefact (l'application) va être poussée vers un repository ou d'autres phases de test peuvent être effectuées.
- Du nettoyage ou autres tâches d'informations post pipeline peuvent être efféctuées.

Orchestration CI/CD avec Jenkins

Jenkins

Jenkins – Pipeline as Code, les scripts

Structure du Pipeline as Code

Pipeline script

```
node('jenkins-slave'){
 container('jnlp'){
  stage('Etape'){
   sh 'java -version‘
  }
 }
}
```

Hôte/pod

Conteneur ‘jnlp’

step sh

java -version

Jenkins – Pipeline as Code, les scripts

Il existe de nombreuses possibilités afin de créer des pipelines toujours plus personnalisés et optimisé notamment grâce aux différentes instructions et plugins qu'on peut passer à Jenkins

Exemple d'instruction groovy

- **Pipeline :** Définit au début d'un script, elle englobe toutes les étapes du pipeline
- **Stage :** Définit une étape générique dans le pipeline qui peut encapsulé plusieurs sous étapes (steps)
- **Steps :** Définit une action spécifique à l'intérieur d'une « stage »
- **Node :** permet de spécifier sur quel agent Jenkins l'étape dois être exécutée
- **Script :** utilisé pour exécuter du code Groovy arbitraire au sein d'un étape (opération avancées ou personnalisation)

Orchestration CI/CD avec Jenkins

Jenkins – Pipeline as Code, les scripts

Exemple d'instruction groovy

- **Parallel :** Permet d'exécuter plusieurs étapes en parallèle (de manière asynchrone)
- **Environment :** Permet de définir des variables d'environnement qui seront disponibles pour toutes les étapes du pipeline
- **Input :** Permet de définir un point d'arrêt interactif où l'utilisateur doit fournir une entrée
- **Container :** permet de définir un conteneur comme environnement d'exécutions de la ou des étapes inclus dans le bloc de code (utile par exemple pour compiler du code depuis un container incluant tout le nécessaire)
- **Post :** Permet de spécifier des actions à effectuer après l'exécution du pipeline (nettoyage, journalisation,…)

Orchestration CI/CD avec Jenkins

Jenkins – Pipeline as Code, les scripts

Le pipeline Jenkins (le processus de build) est décrit dans un fichier texte appelé '**Jenkinsfile**' et persisté dans les sources du projet.

Remarques

- Le processus de build peut être différencié par branche du projet
- Le développeur a la main sur le pipeline et peut le modifier à sa guise
- Le processus est partagé avec les différents membres du projet

Exercice 4

1er Job Jenkins

TP4 – 1er Job sur Jenkins

- Créer un 1er pipeline simple
- Lancer le Job à une étape
- Rajouter des étapes
- Interconnecter Jenkins à GitHub
- Pipeline script from SCM

Exercice 5

Pipeline sequential, parallèle et pipeline successif avec des paramètres

TP5 – Sequentiel / parralel / paramètre

- Créer un pipeline avec tâches séquentielles
- Créer un pipeline avec des tâches parallèles
- Créer un Pipeline avec des paramètres
- Créer un pipeline successif qui récupère les paramètres de son prédécesseur

Exercice 6

Integration de test avec SonarQube

TP6 - SonarQube

- Créer un nouveau projet sur GitHub
- Créer et configurer le Job Jenkins
- Configurer Maven dans Jenkins
- Installer SonarQube et créer le projet
- Configurer Jenkins pour travailler avec SonarQube
- Créer le pipeline de build + test
- Parcourir les tests sur SonarQube